

APUNTESWEB · INGENIERÍA INFORMÁTICA

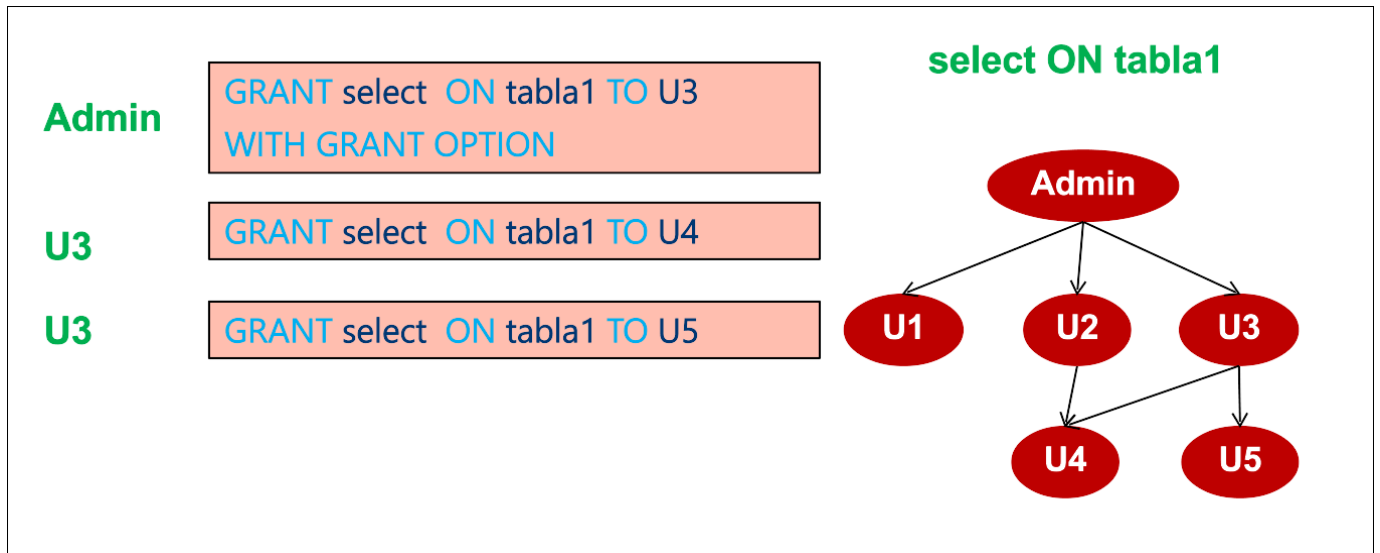


Segundo curso · 2.º cuatrimestre · 2 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

Apuntes Prácticos

1. Seguridad



Se hace revocación en cascada por defecto. \$U4\$ y \$U5\$ mantienen sus permisos porque todavía hay un camino hasta el administrador.

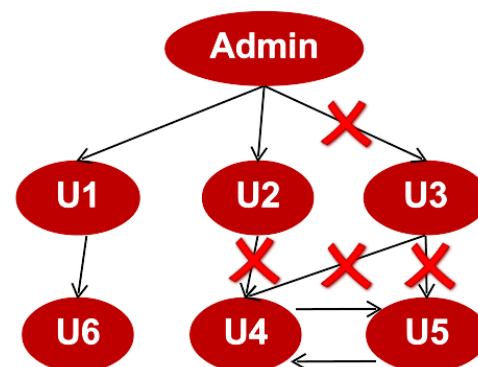


Ahora \$U4\$ y \$U5\$ pierden sus permisos.

```
select ON tabla1
```

Admin REVOKE select ON tabla1 FROM U3

U2 REVOKE select ON tabla1 FROM U4



Esto generaría una **excepción** porque **RESTRICT** es una opción que impide revocar un privilegio si ese usuario ha dado ese privilegio a otros usuarios.

Admin

```
REVOKE select ON tabla1 FROM U1
RESTRICT
```

Si le quitamos la opción de dar privilegios y borramos al usuario \$U6\$ los pierde consecuentemente (los pierde por que se borra, no por quitar el grant option).

■ Revocación de privilegios

Admin REVOKE select ON tabla1 FROM U3

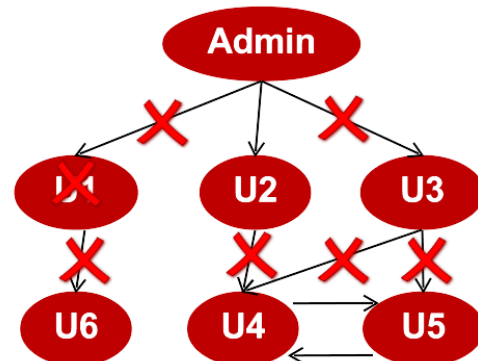
U2 REVOKE select ON tabla1 FROM U4

```
Admin REVOKE select ON tabla1 FROM U1  
RESTRICT
```

```
Admin REVOKE grant option ON tabla1 FROM
U1
```

Admin DROP USER U1

```
select ON tabla1
```



Aquí el permiso lo concede el **rol no el usuario**, por lo que si eliminamos al usuario $U1$, $U6$ sigue manteniendo sus permisos.

Admin

```
CREATE ROLE R1
GRANT select ON tabla1 TO R1 WITH GRANT OPTION
GRANT R1 TO U1
```

U1

```
SET ROLE R1
GRANT select ON tabla1 TO U6 GRANTED BY CURRENT ROLE
```

Admin

```
DROP USER U1
```

2. Gestión de Transacciones

Propiedades ACID:

- Atomicidad
- Consistencia
- Aislamiento
- Durabilidad

Estados de una transacción:

- **Activa**
- **Parcialmente comprometida:** ha ejecutado su última operación
- **Comprometida:** cambios guardados en almacenamiento estable
- **Fallida**
- **Abortada:** retrocedidos los cambios tras pasar por fallida

Secuencialidad

- **Operaciones conflictivas (cada una es de una T diferente):**
 - leer(A) y leer(A) **no** es conflictiva
 - leer(A) y escribir(A) es conflictiva
 - escribir(A) y leer(A) es conflictiva
 - escribir(A) y escribir(A) es conflictiva
- **Planificación secuencial:** se ejecutan todas las operaciones de una misma transacción seguidas en bloque.
- **Planificación secuenciable:** produce mismo resultado que la secuencial pero empleando concurrencia (no tiene por que ser secuenciable en cuanto a conflictos).
- **Planificación secuenciable en cuanto a conflictos:** si se intercambian operaciones no conflictivas se obtiene la planificación secuencial.

T1	T2	T3
Bloquear-C (A)		
Leer (A)		
	Bloquear-X (C)	
	Escribir (C)	
		Bloquear-C (C)
	Bloquear-C (B)	
	Leer (B)	
Bloquear-C (D)		
Leer (D)		
Bloquear-X(B)		
	Desbloquear (C)	
		Leer (C)
		Desbloquear (C)
	Desbloquear (B)	
Escribir (B)		
Desbloquear (A)		
Desbloquear (B)		
Desbloquear (D)		

- **Grafo de precedencia:** permite saber si la **planificación es secuenciable en cuanto a conflictos** (*no tiene ciclos*). $T_i \rightarrow T_j$, si tenemos 2 operaciones conflictivas y T_i ejecuta antes la suya que T_j . En la tabla de arriba tendríamos $T_2 \rightarrow T_1$ y $T_2 \rightarrow T_3$

Aislamiento y Atomicidad

- **Planificación recuperable:** aquella en la que todo par de transacciones T_1 y T_2 donde T_2 lee elementos escritos por T_1 , la operación `commit` de T_1 aparece antes que la `commit` de T_2

T_1	T_2
leer(A)	
escribir(A)	
	leer(A)
leer(B)	
commit	
	commit

- **Planificación sin cascada:** aquella en la que todo par de transacciones T_1 y T_2 donde T_2 lee elementos escritos por T_1 , la operación `commit` de T_1 aparece antes que la `leer` de T_2

T_1	T_2
leer(A)	
escribir(A)	
commit	
	leer(A)
	commit

Niveles de Aislamiento

- **Secuenciable**
- **Lectura repetible:** solo permite leer datos **comprometidos**, e **impide que entre 2 lecturas** de un dato en la transacción otra actualice ese dato
- **Lectura no repetible:** solo permite leer datos **comprometidos**, pero **permite que entre 2 lecturas** de un dato en la transacción, otra actualice ese dato
- **No comprometida:** permite leer datos **no comprometidos**

3. Control de Concurrency

Protocolo de Bloqueo en 2 fases

Tiene dos fases:

- **Fase de crecimiento:** la transacción sólo obtiene bloqueos
- **Fase decrecimiento:** la transacción sólo libera bloqueos
- **Punto bloqueo:** punto de la transacción donde se produce la primera liberación

Protocolo Estricto

Sólo se pueden liberar los `bloqueo-x` una vez se ha **comprometido la transacción**.

Protocolo Riguroso

Sólo se pueden liberar los **bloqueos en general** una vez se ha **comprometido la transacción**.

Protocolo con conversiones de Bloqueo

Emplea 2 operaciones para la conversión de bloqueos

- `subir` **de compartido a exclusivo**
- `bajar` **de exclusivo a compartido** Los desbloques se hacen una vez se compromete o **aborta la transacción**

T1	T2	T3
Bloquear-C (A)		
Leer (A)		
	Bloquear-X (C)	
	Escribir (C)	
		Bloquear-C (C)
	Bloquear-C (B)	
	Leer (B)	
Bloquear-C (D)		
Leer (D)		
Bloquear-X(B)		
	Desbloquear (C)	
		Leer (C)
		Desbloquear (C)
	Desbloquear (B)	
Escribir (B)		
Desbloquear (A)		
Desbloquear (B)		
Desbloquear (D)		

Si suponemos que después de la última operación de cada T_i hay un `commit` cumple tanto el ****protocolo estricto como el riguroso****. Si no, no los cumple.

Prevención de Interbloqueos

Expropiar y retroceder, cuando A solicite un bloqueo que posee actualmente B :

- **Esperar-Morir:**
 - Si $\text{marca}(A) < \text{marca}(B)$, A espera
 - Si $\text{marca}(A) > \text{marca}(B)$, A retrocede
- **Herir-Esperar:**
 - Si $\text{marca}(A) < \text{marca}(B)$, B retrocede
 - Si $\text{marca}(A) > \text{marca}(B)$, A espera

Detección de Interbloqueos

- **Grafo de Espera:** se añadirá una arista $T_i \rightarrow T_j$ cuando la transacción T_i solicite un elemento de datos que posea en ese momento la transacción T_j . Se borra cuando T_j deja de poseer un elemento que necesita T_i . **Si hay ciclo hay interbloqueo.**

T1	T2	T3
Bloquear-C (A)		
Leer(A)		
	Bloquear-C (B)	
	Leer(B)	
Bloquear-X (B)		
		Bloquear-C (A)
		Leer(A)
	Bloquear-X (A)	
	Desbloquear(B)	
Escribir(B)		
Desbloquear(A)		
Desbloquear(B)		

- $T_1 \rightarrow T_2$
- $T_2 \rightarrow T_3$
- $T_2 \rightarrow T_1$ Hay un ciclo

Protocolo de Validación Optimista

Orden de secuencialidad/serialización: determina el orden de las transacciones en función sus **marcas temporales** usando el valor de **validación** como **marca temporal**. Para cumplir la prueba de validación se debe verificar una condición u otra:

- $fin(T_i) < inicio(T_j)$
- $inicio(T_j) < fin(T_i) < validacion(T_j)$ y además $escritos(T_i) \cap leidos(T_j) = \emptyset$

T_1 (lee (A,B), escribe: C), T_2 (lee C, escribe A) y T_3 (lee B, escribe D).

Orden de serialización: $T_1 \rightarrow T_2 \rightarrow T_3$

- **Prueba de validación para T_2 :**
 - $fin(T_1) < inicio(T_2)$, $50 < 20$ falso
 - $inicio(T_2) < fin(T_1) < validacion(T_2)$, $20 < 50 < 60$ verdadero
 - $escritos(T_1) \cap leidos(T_2) = \emptyset$, $C \cap C = \emptyset$ falso, no pasa la prueba
- **Prueba de validación para T_3 :**
 - $fin(T_1) < inicio(T_3)$, $50 < 30$ falso
 - $inicio(T_3) < fin(T_1) < validacion(T_3)$, $30 < 50 < 65$ verdadero

- $escritos(T_1) \cap leidos(T_3) = \emptyset, C \cap B = \emptyset$ verdadero, pasa la prueba

Validación Aislamiento de Instantáneas

- **Primer compromiso gana:** cuando una T está **parcialmente comprometida** se comprueba si hubo alguna transacción concurrente con T que haya hecho alguna actualización sobre los elementos escritos por T
 - Si no se encuentra dicha transacción: T **se compromete**
 - Si se encuentra: T **se aborta**
- **Primera actualización gana:** cuando una T intenta actualizar un elemento de datos, solicita una **bloqueo de escritura sobre él**. Se comprueba si alguna transacción concurrente con T tiene un bloqueo sobre el mismo elemento:
 - Si no la hay:
 - Si ha sido actualizado por otra: T **aborta**
 - Si no ha sido actualizado: T **puede continuar**
 - Si la hay, T espera a que acabe:
 - Si la otra aborta: T **puede continuar**
 - Si la otra se compromete: T **aborta**

Estos métodos no aseguran la secuencialidad pero no importa, funciona igual, el grafo de precedencia puede tener ciclos.

4. Sistema de Recuperación

Memoria Interna (no creo q caiga)

- **Bloque físico** (en disco)
- **Bloque de memoria intermedia** (en memoria intermedia)
- **Memoria intermedia** (área de la RAM)
- **Área de Trabajo de T** (también en RAM y contiene copias de elementos de datos X)
- **Entrada(B):** transfiere B de disco a memoria intermedia
- **Salida(B):** transfiere B de memoria intermedia a disco
- **leer(X):** asigna el valor del elemento X a la variable X_i del **área de trabajo**. Si el bloque B_x donde está X no está en **memoria intermedia** se ejecuta **Entrada(B_x)**
- **escribir(X):** asigna el valor de la variable X_i al elemento X . Si el bloque B_x donde está X no está en **memoria intermedia** se ejecuta **Entrada(B_x)**

- `Salida(B)` se ejecuta cuando necesitamos espacio para meter otro bloque y lo vamos a poner en uno que ha sido escrito en memoria intermedia.

Registro Histórico

`<Ti, E, Vant, Vpost>` `<Ti iniciada>`, `<Ti comprometida>`, `<Ti abortada>`.

Modificación de la Base de Datos

- **Modificación Inmediata:** la T modifica la bd mientras está activa
- **Modificación Diferida:** la T no modifica la bd hasta que está **comprometida**
- `deshacer(T)`: restaura el valor de todos los elementos actualizados por la transacción T a sus valores anteriores.
- `rehacer(T)`: establece el valor de todos los elementos actualizados por la transacción T a los valores nuevos.

Algoritmo de recuperación

- **1º Fase rehacer:** partimos del último **checkpoint**, creamos la **lista-deshacer**. Cuando se encuentra un registro de la forma `<Ti, Xj, V1, V2>` o uno de **solo-rehacer** de la forma `<Ti, Xj, V1>` se aplica `rehacer`. Si se encuentra un registro de la forma `<Ti iniciada>` se añade a la lista-deshacer. Si se encuentra de la forma `<Ti abortada>` o `<Ti comprometida>` se elimina `Ti` de la **lista-deshacer**.
- **2º Fase deshacer:** recorremos desde abajo hasta el **checkpoint**. Cuando veamos un registro de una transacción de la **lista-deshacer** escribimos un **registro de compensación** de la forma `<Ti,Xj,V1>` (**SIEMPRE QUE NO USEMOS MODIFICACIÓN DIFERIDA**). Al encontrar un registro de la forma `<Ti iniciada>` escribimos `<Ti abortada>` y se elimina de la **lista-deshacer**. Repetimos esto hasta que la **lista-deshacer** está vacía.

Fallo en el tiempo 12. $A = 20$ y $B = 60$

Con modificación inmediata: 1º Se rehace todo de arriba a abajo. Añadimos a T_1 a la **lista-deshacer**, después la sacamos y añadimos a T_2 2º Aplicamos deshacer a T_2 de abajo hacia arriba, escribimos un registro de solo rehacer y escribimos transacción abortada

<i>Valor A</i>	<i>Valor B</i>	<i>Registro</i>	<i>Tiempo</i>
20	60	<code><T1 iniciada></code>	1
20	60		2
20	10	<code><T1,B,60,10></code>	3
20	10		4
20	10		5
20	10	<code><T1,B,60,10></code>	6

<i>ValorA</i>	<i>ValorB</i>	<i>Registro</i>	<i>Tiempo</i>
20	10		7
20	10	<T1 comprometida>	8
20	10	<T2 iniciada>	9
20	10		10
10	10	<T2, A, 20, 10>	11
20	10	<T2,A,20>	12
20	10	<T2 abortada>	13

Con modificación diferida: 1° Se rehace todo de arriba a abajo. Añadimos a T_1 a la **lista-deshacer**, después la sacamos y añadimos a T_2 2° Aplicamos deshacer a T_2 de abajo hacia arriba, escribimos un registro de solo rehacer y escribimos transacción abortada

<i>ValorA</i>	<i>ValorB</i>	<i>Registro</i>	<i>Tiempo</i>
20	60	<T1 iniciada>	1
20	60		2
20	60	<T1,B,10>	3
20	60		4
20	60		5
20	60	<T1,B,10>	6
20	60		7
20	10	<T1 comprometida>	8
20	10	<T2 iniciada>	9
20	10		10
20	10	<T2, A, 10>	11
20	10	<T2 abortada>	12

5. Indexación y Asociación

Hashing Estático

Cajón (bucket): unidad de almacenamiento de tamaño fijo que puede guardar uno o más registros. Para indexar en estos cajones los registros usamos funciones hash.

Si los cajones se llenan añadimos **punteros a cajones de desbordamiento:**

Podemos:

- **Buscar**
- **Insertar**
- **Borrar**, en principio los cajones aunque sean de desbordamiento se quedan vacíos, no desaparecen.

Índices Asociativos

Lo mismo que antes pero en vez de guardar en los cajones registros guardamos índices que apuntan a registros.

Hashing Dinámico

Partimos de esto, donde el \$0\$ de la izquierda es la **profundidad global** y el \$0\$ de la derecha es la **profundidad local**. Si llenamos el cajón y se produce un desbordamiento hay que tener en cuenta lo siguiente: - Si la **profundidad local** es menor que la **profundidad global** no pasa nada, simplemente se crea un nuevo cajón y se enlaza con la tabla de la izquierda sin mayor complicación. Además aumentamos la profundidad local de cada cajón. - Si la **profundidad local** es igual que la **profundidad global**, habrá que aumentar tanto la profundidad global como la **local** en los cajones implicados, duplicando el tamaño de la tabla, creamos otro cajón y lo enlazamos. Ambas implican reordenar los elementos de los cajones y los punteros a los cajones. Sebastián para indexar usa los **msb**.

- Insertamos a mozart (\$0011\$), y va al único bucket que hay. - Insertamos a Srinivasan(\$1111\$), y va al bucket que hay. - Sin embargo ahora insertamos a Wu (\$1010\$), se desborda, comparamos profundidades y dividimos, redistribuimos nuevamente los elementos. Además incrementamos las profundidades

- Insertamos a Einstein(\$1001\$), se vuelve a desbordar, como la profundidad local de ese cajón es menor, se crea uno nuevo se reordenan los punteros y se redistribuyen los elementos nuevamente.

- Hay ocasiones en las que debemos añadir un cajón de desbordamiento, cuando insertemos varios elementos que tengan la misma clave de inserción y superen el tamaño máximo del bucket. Por lo que simplemente se haría como al fondo de la imagen

6. Optimización de Consultas

1. Reordenar joins para reducir el tamaño de relaciones intermedias.
2. Aplicar proyecciones tempranas para eliminar atributos innecesarios.

Información Útil

De esta asignatura me dió pereza hacer apuntes. El examen de recuperación puso cosas de optimización de consultas y cosas muy raras.

Tema 1

- Saber por qué y como accedemos a las bd desde lenguajes de programación gnerales
- JDBC
- SQL incorporado y lo de los cursores

Tema 2

- Saber como conceder privilegios
- Saber lo que es un grupo y un rol
- Saber como se transfieren los privilegios

Tema 3

- Saberse muy bien las propiedades ACID
- Saberse muy bien los estados de una Transacción
- Saber todo lo de secuenciable en cuanto a conflictos teórica y prácticamente

Tema 4

- Saberse muy bien todos los protocolos de bloqueo, teórica y prácticamente
- Saberse muy bien lo del aislamiento de instantáneas teórica y prácticamente
- Saberse muy bien los protocolos basados en validación, teórica y prácticamente

Tema 5

- Saberse los tipos de fallos del sistema de recuperación
- Saberse muy bien como restaurar la base de datos, mirando los algoritmos y lo del registro histórico, de forma teórica y prácticamente

Tema 6

- Saber de forma teórica como se organiza todo en archivos
- Lo de gestión del buffer y esas historias del final yo creo que no es muy importante

Tema 7

- Muy importante saber como funcionan los árboles b+ y las ventajas que tienen frente al hashing.
- Saber lo de los índices
- Saberse muy bien el hashing estático y dinámico
- Saber de forma práctica como hacer las tablas de un hashing dinámico
- Recomendable ver vídeos de indios en youtube

Tema 8

- Si llegas a este tema y eres capaz de retener algo más pues míralo en general un poco por encima, pero no merece la pena estudiarse todos los algoritmos esos ni todas las operaciones, solo saber como funciona por encima todo.